

The Saga Pattern in Microservices

Microservices（微服务）

概念：Microservices 是一种将软件设计为一套独立部署的服务的方法，通常围绕业务功能，比如银行和金融机构需要处理复杂的交易处理、客户管理、风险控制等功能。微服务架构可以帮助将这些功能分离成不同的模块，提升系统的安全性、可扩展性和维护性。例如，将账户管理、交易处理、风控系统分开部署，方便独立升级和维护。

如果你想做交易并且必须在不同模块之间进行交互，你可以在同一个系统中使用所有这些模块，并且还可以在它们之间共享一个公共数据库。然后我们可以进入一个微服务架构，其中每个模块都可能是不同的服务，当您拥有整体应用程序或单个应用程序时，您可以轻松地在不同模块之间进行协调。它们共享同一个数据库，您可以将事务放在其中，也可以将事务放在应用程序级别上，但现在我们处于微服务架构中。

Saga 模式

Saga 是一种分布式事务管理模式，用于确保跨多个微服务的业务流程的一致性。在 Saga 模式中，分布式事务被分解为一系列独立的小事务，每个小事务由不同的微服务负责。如果某个步骤失败，系统会依次执行相应的补偿操作来撤销已完成的步骤。Saga 模式基本上

提供了一种以类似事务的方式在不同服务之间进行通信的方法。你永远不会获得隔离，但你可以让它看起来像当您只有一个数据库或使用同一服务时进行的资产交易。在 **saga** 模式中，您在不同的服务中执行一系列事务。每个服务都可以执行一个事务，并尝试使其看起来只有一个事务。如果出现故障或任何服务或每个服务中的任何事务服务，然后你尝试用其他行动来补偿。

场景描述：银行转账

假设我们有一个银行系统，包含以下微服务：

账户服务（**Account Service**）：管理用户账户和余额。

转账服务（**Transfer Service**）：处理转账请求。

通知服务（**Notification Service**）：发送转账通知。

转账流程

用户通过 **API** 网关提交转账请求（从账户 **A** 转账到账户 **B**）。

转账服务验证请求并创建转账记录，状态为“处理中”。

转账服务请求账户服务扣减账户 **A** 的余额。

账户服务扣减账户 **A** 的余额成功后，转账服务请求账户服务增加账户 **B** 的余额。

账户服务增加账户 **B** 的余额成功后，转账服务更新转账记录状态为“已完成”，并通知通知服务发送成功通知。

如果在任何步骤中发生失败，系统将执行相应的补偿操作来撤销已完成的步骤。

使用 Saga 模式实现

1. 基于事件的 Saga 实现

各个服务通过消息队列（如 Kafka、RabbitMQ）进行通信和触发事件。

转账服务：

- 创建转账记录（Create Transfer）
- 发送扣减账户 A 余额消息（Debit Account A）
- 接收账户服务的扣减确认消息或失败消息
- 发送增加账户 B 余额消息（Credit Account B）
- 接收账户服务的增加确认消息或失败消息
- 发送通知消息（Send Notification）

账户服务：

- 接收扣减余额消息
- 扣减余额成功，发送确认消息（Account Debited）
- 扣减余额失败，发送失败消息（Debit Failed）
- 接收增加余额消息
- 增加余额成功，发送确认消息（Account Credited）
- 增加余额失败，发送失败消息（Credit Failed）

通知服务：

- 接收通知消息并发送通知

补偿操作：

- 如果扣减账户 A 余额失败，转账服务将发送取消转账消息（Cancel Transfer）
- 如果增加账户 B 余额失败，转账服务将发送补偿账户 A 余额消息（Compensate Account A）

服务编排（Service Orchestration）

服务编排是一种集中控制的方式，由一个编排器（协调器）来管理和调用各个微服务，以完成一个完整的业务流程。编排器负责处理业务逻辑的控制流，确保各个服务按照预期的顺序执行，并处理错误和补偿。

实现方式：

编排器：在服务编排模式中，一个中心编排器负责管理整个业务流程。它通过调用各个微服务的 API 来执行步骤，并根据需要处理错误和补偿。

业务流程：编排器会按照预定义的业务流程依次调用各个服务。例如，在银行转账流程中，编排器会首先调用转账服务创建转账记录，然后依次调用账户服务扣减和增加余额，最后调用通知服务发送通知。

错误处理：在任何一步出现错误时，编排器会执行补偿操作。例如，如果在增加目标账户余额时失败，编排器会调用账户服务补偿源

账户的余额。

集中控制：业务逻辑的控制流集中在编排器中，所有的服务调用和错误处理逻辑都由编排器管理。

示例：银行转账流程

步骤 1：编排器接收用户的转账请求，创建转账记录，状态设为“处理中”。

步骤 2：编排器调用账户服务，扣减账户 A 的余额。如果成功，编排器继续执行下一步；如果失败，编排器执行补偿操作（取消转账）。

步骤 3：编排器调用账户服务，增加账户 B 的余额。如果成功，编排器更新转账记录状态为“已完成”，并调用通知服务发送通知；如果失败，编排器执行补偿操作（补偿账户 A 的余额）。

事件溯源（Event Sourcing）

事件溯源是一种保存应用状态的模式，应用的状态是通过一系列事件（Event）来表示和构建的。这些事件记录了对应用状态的每一次变更，事件是不可变的，通过重放这些事件可以重新构建应用的当前状态。

实现方式：

1.事件存储：在事件溯源模式中，系统的状态变化通过事件来记录。每个事件代表一次状态变更，这些事件被持久化到事件存储中。

2.事件发布：当状态发生变化时，系统生成一个事件，并将其发布到事件存储。例如，在订单处理流程中，当订单创建时，系统生成一个 `OrderCreated` 事件，并将其保存到事件存储中。

3.事件重放：系统的当前状态是通过重放所有相关的事件来构建的。例如，重放 `OrderCreated` 事件可以重新构建订单的初始状态。

4.事件处理器：不同的微服务订阅并处理这些事件，执行相应的业务逻辑。例如，库存服务订阅 `OrderCreated` 事件，当接收到该事件时，执行库存扣减逻辑。

5.一致性和恢复：通过重放事件日志，可以确保系统的一致性，并在发生故障时恢复系统状态。

示例：订单处理流程

步骤 1：订单服务接收用户的订单请求，生成 `OrderCreated` 事件，并将其保存到事件存储中。

步骤 2：库存服务订阅 `OrderCreated` 事件，当接收到该事件时，执行库存扣减操作，并生成 `InventoryReserved` 事件。

步骤 3：支付服务订阅 `OrderCreated` 事件，当接收到该事件时，执行支付处理操作，并生成 `PaymentProcessed` 事件。

步骤 4：订单服务订阅 `InventoryReserved` 和 `PaymentProcessed` 事件，更新订单状态为“已完成”。

总结

服务编排：

- 提供集中控制的业务流程管理，由一个编排器管理服务调用和错误处理。
- 适用于复杂业务流程需要明确控制的场景。

事件溯源：

- 提供事件驱动的状态管理，通过事件记录和重放构建系统状态。
- 适用于需要完整历史记录和复杂状态恢复的系统。